

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো
(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাস
(Indian Institute of Technology, Madras)

সপ্তাহ – 07

বক্তৃতা – 30

ডাইনিং ফিলসোফার প্রব্লেম
(Dining Philosophers Problem)

হ্যালো। পূর্ববর্তী ভিডিওতে, আমরা semaphores সম্পর্কে বললাম। আমরা দেখেছি কিভাবে producer-consumer সমস্যা সমাধানের জন্য semaphore ব্যবহার করা যেতে পারে। এই ভিডিওতে, আমরা অন্য একটি সমস্যা দেখব, যেখানে semaphores দরকারী। সুতরাং, এটি Dining Philosophers Problem এবং এটা semaphores ব্যবহারের একটি চমৎকার উদাহরণ।

(স্লাইড সময় : 00:54 দেখুন)

তাই আসুন সমস্যাটি শুরু করি। আমাদের পাঁচটি philosopher P1, P2, P3, P4, P5 টেবিলের চারপাশে বসে আছে। এখন তাদের সামনে, প্রত্যেকের জন্য পাঁচটি প্লেট আছে; philosopher এবং 1,2,3,4,5 পাঁচটি fork আছে। এখন এই প্রত্যেক philosopher দুটো বিষয়ের মধ্যে একটি মাত্র করতে পারেন। প্রতিটি philosopher ভাবতে বা খেতে পারেন। অন্য কথায়, যদি দার্শনিক চিন্তা না করেন তবে তিনি খান এবং তদ্বিপরীত। এখন, খাওয়ার জন্য, একজন philosopher কে উভয় fork দরকার, তার বামের fork এবং ডানদিকের fork, উদাহরণস্বরূপ, যদি P1 খেতে চান তবে তাকে fork 1 এবং fork 2 নিতে হবে। এই দুটি fork যা তার নিকটে আছে। একইভাবে, P3

এর প্রয়োজন fork 4 এবং fork 3 যদি তিনি খেতে চান। এখন সমস্যাটি বা আমরা যা সমাধান করার চেষ্টা করছি তা হল একটি algorithm তৈরি করা, যেখানে কোনও philosopher ক্ষুধার্ত থাকে না, প্রত্যেক দার্শনিকই অবশেষে খেতে সুযোগ পায়।

(স্লাইড সময় 02:30 দেখুন)

সুতরাং আসুন আমরা এই বিশেষ সমস্যার খুব সহজ সমাধান দিয়ে শুরু করি। সুতরাং আসুন আমাদের এখানে একটি সমাধান আছে বলা যাক, আমরা N যেটা এখানে 5 হিসাবে প্রত্যেক philosopher কে সংজ্ঞায়িত করি। এবং আমরা philosopher এর জন্য একটি ফাংশন আছে। সুতরাং, এই ফাংশনটি একটি পূর্ণসংখ্যা মান 'i' নেয় এবং এই 'i' 1 থেকে 5 এর মান হতে পারে যে প্রত্যেকটি philosopher P1, P2, P3, P4, P5 এর সাথে মিলবে। এখন ফাংশনে আমরা একটি infinite loop নিই যেখানে i তম philosopher কিছু সময় ভাবেন এবং তারপর কিছু সময় পরে তিনি ক্ষুধার্ত বোধ করবেন। সুতরাং, তিনি ডানদিকের fork টি গ্রহণ করবেন, তারপর তার বামে fork টি নেবেন, তারপর তিনি কিছু সময়ের জন্য থাকবেন, এবং পরে তিনি বাম fork টি নিচে রাখবেন এবং তারপর ডান fork টি নিচে রাখবেন, এবং এটি অসীমভাবে একটি লুপ। সুতরাং, উদাহরণস্বরূপ philosopher P1 কিছু সময়ের জন্য চিন্তা করবে, তারপর ক্ষুধার্ত বোধ করবে, তারপর তিনি ডান দিকের fork 1 টি নেবেন, তারপর বাম দিকের fork 2 নেবেন, তারপর কিছু সময়ের জন্য থাকবেন এবং তারপরে fork গুলি নামিয়ে রাখবেন, প্রথম 2 এবং তারপর 1 নামিয়ে রাখবেন। সুতরাং, এটা সমস্যাটির একটি খুব সহজ এবং সহজ সমাধান মত মনে হচ্ছে। কিন্তু আমরা দেখব যে কিছু সমস্যা আছে যা আসতে পারে। সমস্যাটি আসে কারণ প্রতিটি philosopher এ মূলত এই ফাংশনটি স্বাধীনভাবে চালান, বা চিন্তা করেন, এবং ক্ষুধার্ত হন এবং খান সব স্বাধীনভাবে। সুতরাং আসুন আমরা এই বিশেষ পরিস্থিতি বিবেচনা করি। ধরুন যে philosopher P1 এবং P3 এর একটি উচ্চতর priority রয়েছে যে যখনই fork এর জন্য অনুরোধ করা হয় তখন সিস্টেমটি সবসময় নিশ্চিত করে যে তারা fork পান। তাই, এই ঘটনায় কি ঘটবে? তাই আমরা এমন একটি অবস্থা পেতে পারি যেখানে P1 যখন ইচ্ছা করে তখনই খেতে পারেন, এবং P3 যখন ইচ্ছা করে তখনই খেতে পারেন, আর অন্য philosopher P2, P4 এবং P5 যাদের fork বাছাই করতে কম priority তারা খেতে পাবেন না। উদাহরণস্বরূপ, philosopher P2 ডান fork বা বাম fork বাছাই করতে পারেন না কখনই, এবং সেইজন্য, P2 খেতে পাবেন না। একইভাবে, P4 এবং P5 তাদের মধ্যে কেবল একটি fork আছে, যা সম্ভবত তারা নিতে পারেন; প্রতিটি ক্ষেত্রে অন্য fork টি philosopher P1 এবং P3-এ উচ্চ probability তে দেওয়া হবে। সুতরাং P2, P4 এবং P5 ক্ষুধার্ত হবে; এবং এটা আমাদের সমস্যার জন্য আদর্শ সমাধান নয়।

(স্লাইড সময় 05:56 দেখুন)

সুতরাং আসুন আমরা আরেকটি সম্ভাব্য সমস্যা দেখতে পারি যা হতে পারে। ধরি সব philosopher একসঙ্গে তাদের ডান fork নিলেন। সুতরাং, philosopher P1 তার ডান fork অধিকার করেছেন, philosopher P2 ডান fork অধিকার করেছেন, P3 ডান fork অধিকার করেছেন, P4, P5 ডান fork অধিকার করেছেন। এখন, প্রত্যেকের খাওয়ার জন্য, ডান fork অধিকার করেছেন, philosopher দেব তাদের বাম fork নেওয়ার দরকার, ফলে starvation হতে পারে। মূলত P1 fork এর জন্য P2 এর অপেক্ষা করছে, যাতে সে তা তুলে নিতে পারে। তারপর P2 fork এর জন্য P3 এর অপেক্ষা করছে,; P3 fork এর জন্য P4 এর অপেক্ষা করছে,; P4, P5 এর অপেক্ষা করছে,; এবং P5 fork এর জন্য P1 এর অপেক্ষা করছে। তাই মূলত আমরা দেখতে পাচ্ছি যে প্রত্যেক দার্শনিক অন্য দার্শনিকের জন্য অপেক্ষা করেছেন, এইভাবে একটি cycle তৈরি করেছেন। এবং এই অপেক্ষা অসীম হবে ফলে starvation হবে, যাকে আমরা deadlock বলি।

(স্লাইড সময় 07:19 দেখুন)

সুতরাং, একটি deadlock এর আনুষ্ঠানিক সংজ্ঞা দিতে গেলে; deadlock একটি পরিস্থিতি, যেখানে প্রোগ্রামগুলি কোনো progress ছাড়াই অনির্দিষ্টকালের জন্য চলতে থাকে। প্রতিটি প্রোগ্রাম একটি ঘটনার জন্য অপেক্ষা করছে যা অন্য প্রোগ্রাম বা প্রক্রিয়া হতে পারে। সুতরাং, আপনি এই ক্ষেত্রে দেখুন প্রতিটি philosopher একটি নির্দিষ্ট ঘটনার জন্য অপেক্ষা করছে, যা অন্য philosopher করবে। সুতরাং, একটি circular wait উপস্থিত এবং deadlock আছে ফলে starvation হয়।

(স্লাইড সময় 08:07 দেখুন)

তাহলে আসুন আমরা এই বিশেষ সমস্যার সমাধান করার অন্য চেষ্টাটি দেখি। সুতরাং আসুন আমরা আমাদের এখানে একই ফাংশন আছে বলে যাক। এবং এই সময় philosopher ডান fork নেয়, তারপর তিনি বাম fork পাওয়া যায় কিনা তা নির্ধারণ করবে; যদি বাম fork পাওয়া যায় তবে philosopher বাম fork টি গ্রহণ করবে, কিছুক্ষণ ধরে থাকবে এবং ডান ও বাম fork দুটোই নামিয়ে রাখবে এবং লুপে চলতে চলবে। যাইহোক, যদি বাম fork পাওয়া না যায়, তাহলে আমরা else অংশে যেতে পারি এবং philosopher ডান fork নামিয়ে রাখবে। মূলত, এখানে তোলা fork, ডান fork টি টেবিল ফিরে আসবে, যদি philosopher দেখে যে বাম fork উপলব্ধ নয়। তাই আর ফলে অন্য philosopher সম্ভবত খাওয়া শুরু করবে। এবং পরে philosopher কিছু সময় T পর্যন্ত sleep করে আবার চেষ্টা করে। তাই আসুন দেখি এই বিশেষ ক্ষেত্রে কি সমস্যা? সুতরাং আসুন একটি নির্দিষ্ট scenario বিবেচনা করুন যেখানে সমস্ত philosopher একই সময়ে শুরু হয়, তারা একযোগে চালায় এবং ঠিক একই সময়ে চিন্তা করে। সুতরাং, এটি একটি পরিস্থিতি হতে পারে যেখানে philosopher একযোগে তাদের fork

তুলে নেয়, তারপর দেখে যে তাদের বাম fork পাওয়া যায় না, তাই তারা একসাথে তাদের fork গুলি নামিয়ে রাখে, তারপর তারা কিছু সময়ের জন্য sleep করে এবং তারপর তারা প্রক্রিয়া পুনরাবৃত্তি করে। সুতরাং, আপনি দেখতে পান যে পাঁচজন philosopher আবার ক্ষুধার্ত করে। তারা ক্রমাগত তাদের ডান fork টি তুলবে এবং টেবিলে নামিয়ে রাখবে। তাই, এই সমাধানটিও আমাদের উদ্দেশ্য সমাধান করতে যাচ্ছে না, যেহেতু আমরা philosopher দেব আবারও ক্ষুধার্ত করছি।

(স্লাইড সময় 10:26 দেখুন)

এই ক্ষেত্রে একটি সামান্য ভাল সমাধান যেখানে একটি নির্দিষ্ট সময়ের জন্য sleep এর পরিবর্তে, philosopher ডান fork টি নিচে নামিয়ে random পরিমাণ সময়ের জন্য sleep করে। যদিও এটি নিশ্চয়তা দেয় না যে starvation ঘটবে না, এটি starvation এর সম্ভাবনাকে হ্রাস করে। এখন এই সমাধান ethernet এর মত protocol এর মধ্যে প্রয়োগ করা হয়।

(স্লাইড সময় 10:57 দেখুন)

তাহলে আসুন আমরা এই বিশেষ সমস্যার সমাধান করার জন্য তৃতীয় প্রচেষ্টাটি দেখি। সুতরাং, এই বিশেষ সমাধান একটি mutex ব্যবহার করে। ডান বা বাম fork গ্রহণের আগে মূলত, দার্শনিককে একটি mutex লক করতে হবে। এবং খাওয়ার পরে এই mutex শুধুমাত্র unlock হয় এবং fork গুলি টেবিলের উপর ফিরে আসে। সুতরাং, fork গুলি নির্বাচন করার আগে এখানে একটি lock mutex আছে এবং fork টেবিলের উপর রাখার পরে একটি unlock mutex আছে। সুতরাং, এই সমাধান মূলত যে starvation ঘটবে না তা নিশ্চিত করে, এটি deadlock কে বাধা দেয়। যাইহোক, এখানে সমস্যা হল যে আমরা একটি mutex ব্যবহার করছি কারণ, তাই অধিকাংশ এক philosopher এই critical section প্রবেশ করতে পারেন। অন্য কথায়, কোনও বিশেষ সময়ে কোন একজন philosopher থেতে পারেন। সুতরাং, যদিও এই সমাধান কাজ করে, এটি সবচেয়ে কার্যকর সমাধান নয়। তাই, আমরা এমন কিছু চাইব যা এর চেয়ে অনেক ভালো।

(স্লাইড সময় 12:20 দেখুন)

সুতরাং আসুন আমরা আমাদের চতুর্থ প্রচেষ্টাটি দেখি, এবং এটি semaphores ব্যবহার করে। তাই আসুন আমরা বলতে পারি যে আমাদের N টি semaphores আছে; তাই semaphores গুলি $s[1]$ থেকে $s[n]$ এবং philosopher প্রতি একটি semaphore আছে। তাই এই সব semaphores প্রাথমিক মান 0 থেকে হয়; উপরন্তু philosopher এর 3 টি state আছে – hungry, eating এবং thinking। তাই, সময়ের কিছুকালের মধ্যে, প্রতিটি philosopher এইগুলির মধ্যে একটি state এ চলে যাবে। উদাহরণস্বরূপ যখন philosopher ভাবছেন, state টি thinking; তারপর philosopher ক্ষুধার্ত হয়ে যায়, তাই এটি hungry state এবং তারপর eating state, শেষে thinking

state এ ফিরে যায়, এবং এই প্রক্রিয়া অনন্তকাল পর্যন্ত অব্যাহত থাকে। সুতরাং, এখানে যে সাধারণ সমাধানটি আমরা দেখছি তা হল যে একজন philosopher কেবল eating state যেতে পারেন যদি না কোনও পার্শ্ববর্তী philosopher eating state এ থাকে। যে একটি philosopher eating state এ যেতে পারে শুধুমাত্র যদি তার বাম এবং তার ডান প্রতিবেশী eating state এ না থাকে। সুতরাং, এই বিশেষ সমাধান বাস্তবায়ন করার জন্য, আমাদের চারটি function আছে। সুতরাং, প্রথম philosopher i অসীম লুপে আছে। তাই philosopher চিন্তা করবে, তারপর fork তুলবে, তারপর কিছু সময়ের জন্য থাকবে এবং fork নামিয়ে রাখবে এবং এই পুনরাবৃত্তি ক্রমাগতভাবে হবে। এখন, takeforks ফাংশনে প্রথমে আমরা philosopher কে hungry state এ রাখি। Philosopher state hungry সেট করা হয় তারপর বলা হয় function test চালু করা হয়। সুতরাং, test কি করে, যে এটি philosopher state hungry কিনা দেখে এবং পাশাপাশি বাম এবং ডান পাশের philosopher এর state eating নয় তা পরীক্ষা করে। যদি সত্যি হয় তবে philosopher খেতে পারেন। এবং শেষে, খাওয়া পরে, fork গুলি নিচে রাখা হয় এবং philosopher thinking state এ যায়।

(স্লাইড সময় 15:06 দেখুন)

তাহলে আসুন, এটি বিস্তারিতভাবে দেখি। আমাদের এই পাঁচটি দার্শনিক P1, P2, P3, P4, এবং P5। এবং আমাদের শুরুতে বলি যে তাদের সমস্ত thinking state এ রয়েছে সুতরাং এটি T দ্বারা এখানে নির্দেশ করে। এবং আমরা জানি তাদের semaphores জন্য প্রাথমিক মান 0 হয়। সুতরাং ধরা যাক সব philosopher thinking state এ আছে। এবং তারপর philosopher 3 thinking state থেকে যায় এবং এটি fork গ্রহণ করে এবং ফলস্বরূপ philosopher 3 জন্য fork ফাংশন আহুত হয়, এবং philosopher thinking state থেকে hungry state এ পরিবর্তিত হয় তারপর ফাংশন test চালানো হয়, এবং এই অবস্থা চেক করা হয়; মূলত philosopher 3 এর state hungry। তাই এই শর্তটি true হলে বাম philosopher P2 এর অবস্থা eating হয় না, কারণ P2 thinking state এ থাকে পাশাপাশি ডান philosopher P4 eating state এ নেই কারণ P4 ও thinking state এ থাকে। তাই, ফলস্বরূপ, এই শর্তটি সত্য হয়ে যায় এবং philosopher P3 এর জন্য state eating নির্ধারিত হয়, তারপর সংশ্লিষ্ট semaphore 0 থেকে 1 পর্যন্ত বাড়ানো হয়। তাই test ফাংশনটি রিটার্ন করে এবং mutex আনলক করে যা, এবং তারপর সেখানে থাকে down হয়। down আমরা জানি s[i] এর মান পরীক্ষা করে, এবং যদি এই মানটি 0 এর চেয়ে কম বা সমান হয় তবে এটি block করা বা অসীম লুপে চলে যাচ্ছে। এবং যদি মানটি 0 এর চেয়ে বড় হয়, তবে এখানে যে কেসটি আছে, তারপর এটি কেবল s[i] এর মানকে হ্রাস করে, সুতরাং এখানে s[i] এর মান 1 ছিল, তাই down block করবে না, বরং এটি শুধু s[i] এর মান হ্রাস করতে যাচ্ছে 0 তে। সুতরাং, s[i] বা সংশ্লিষ্ট semaphore এর 0 মান আছে। এবং তারপর philosopher 3 eating state এ যান এবং খেতে পারেন।

(স্লাইড সময় 17:35 দেখুন)

এখন, আসুন আরেকটি পরিস্থিতি দেখুন, যেখানে philosopher 4 thinking state থেকে take forks state এ যাচ্ছে। এখন philosopher 4 এর take forks invoke হয়, এবং একটি লক mutex আছে, philosopher 4 এর state hungry সেট করা হয় এবং test i চালু করা হয়। সুতরাং, test i philosopher 4 এর জন্য প্রথম শর্ত পূরণ হয়, কারণ P4 যথার্থই hungry state এ আছে,; যাইহোক, eating state টি বাকি আছে। অতএব, যদি এই if condition false হয়, সেইজন্য execution টি if block এ যায় না, বরং একে skip করে। এখন আমরা unlock এবং down এ ফিরে যাই। তাই এখন আমরা কি দেখতে পাচ্ছি P4 এর semaphore এর মান 0 হয়। সুতরাং, যখন আমরা জানি যে down প্রয়োগ করা হয়, এটি প্রক্রিয়াটি block করা শুরু করবে। সুতরাং, এইভাবে philosopher P4 blocked থাকবে, তাই যতক্ষণ P3 এর খাওয়া না হয় P4 blocked থাকবে। তারপর একটি সময় P3 fork টি নিচে রাখার সিদ্ধান্ত নেয়, এবং thinking state এ ফিরে যায়। তারপর এটি বাম philosopher P2 সঙ্গে test চালু হবে, যা আমাদের ক্ষেত্রে সত্যিই আকর্ষণীয় না তাই আমরা এটি দেখব না। কিন্তু যেটা আকর্ষণীয় হয় সেটি test right সঠিক এবং test right এখানে P4 এর অনুরূপ তাই এটি এখানে invoke করা হবে। এবং আমরা যা দেখতে পাই তা হল P4 এর অবস্থা, তাই মনে রাখবেন যে এই test right টি invoke হবে যেখানে i এর মান 4। তাই আমরা দেখতে পাচ্ছি যে i এর state hungry কারণ P4 hungry। বাম এবং ডান থাকছে না, তাই, এই শর্ত true এবং execution কার্যকর হবে এই if এর মধ্যে। ফলস্বরূপ, philosopher 4 এর জন্য state eating হবে এবং semaphore মান P4 এর জন্য 1 এ সেট করা হবে। এখন semaphore মান 1 এর জন্য wake up হবে। তাই P4 যা semaphore এ blocked করা হয়েছিল জেগে উঠবে। এবং এর ফলে semaphore এর মান 0 তে হ্রাস পায়। তাই philosopher P4 জেগে ওঠে এবং খেতে শুরু করে। এইভাবে, আমরা দেখতে পাই যে semaphores নিশ্চিত করেছে যে philosophers পাঁচটি fork share করে নিতে পারে, যা তাদের মধ্যে common। এবং এটি নিশ্চিত করবে যে প্রত্যেক philosopher অবশেষে খাওয়া পাবেন। তাই, Dining Philosopher সমস্যার জন্য অন্যান্য সমাধান রয়েছে যা পড়তে খুবই আকর্ষণীয় তবে আমরা এই বিশেষ কোর্সের জন্য সেখানে যাব না। ধন্যবাদ।